



Industrial Machine Learning on Hadoop and Spark

Maks Nakhodnov, Bremen 2025

Distributed computing. Spark

Plan:

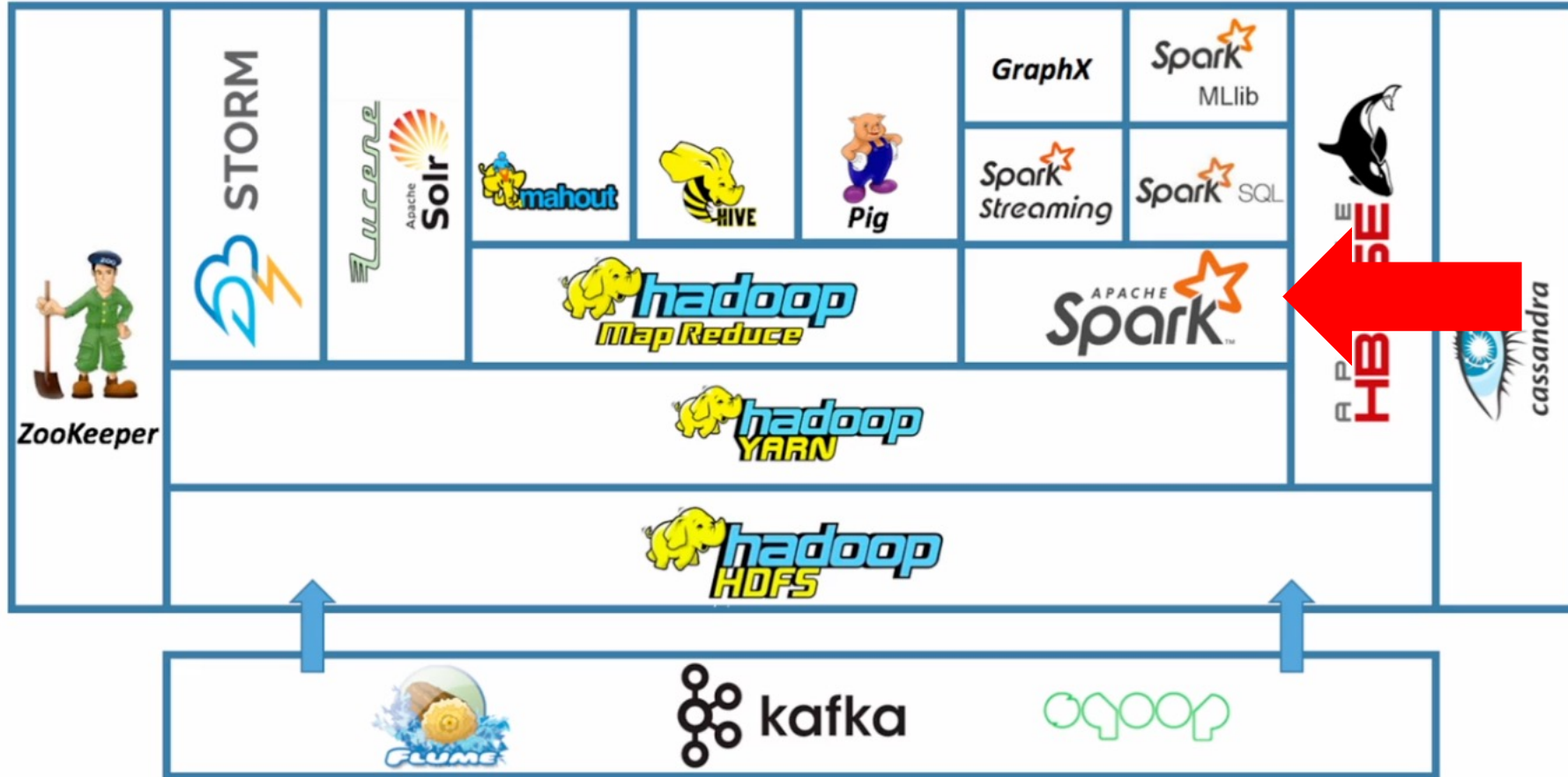
- Spark — main abstractions and objects
- Actions and transformations
- RDD API
- DataFrame API

Spark

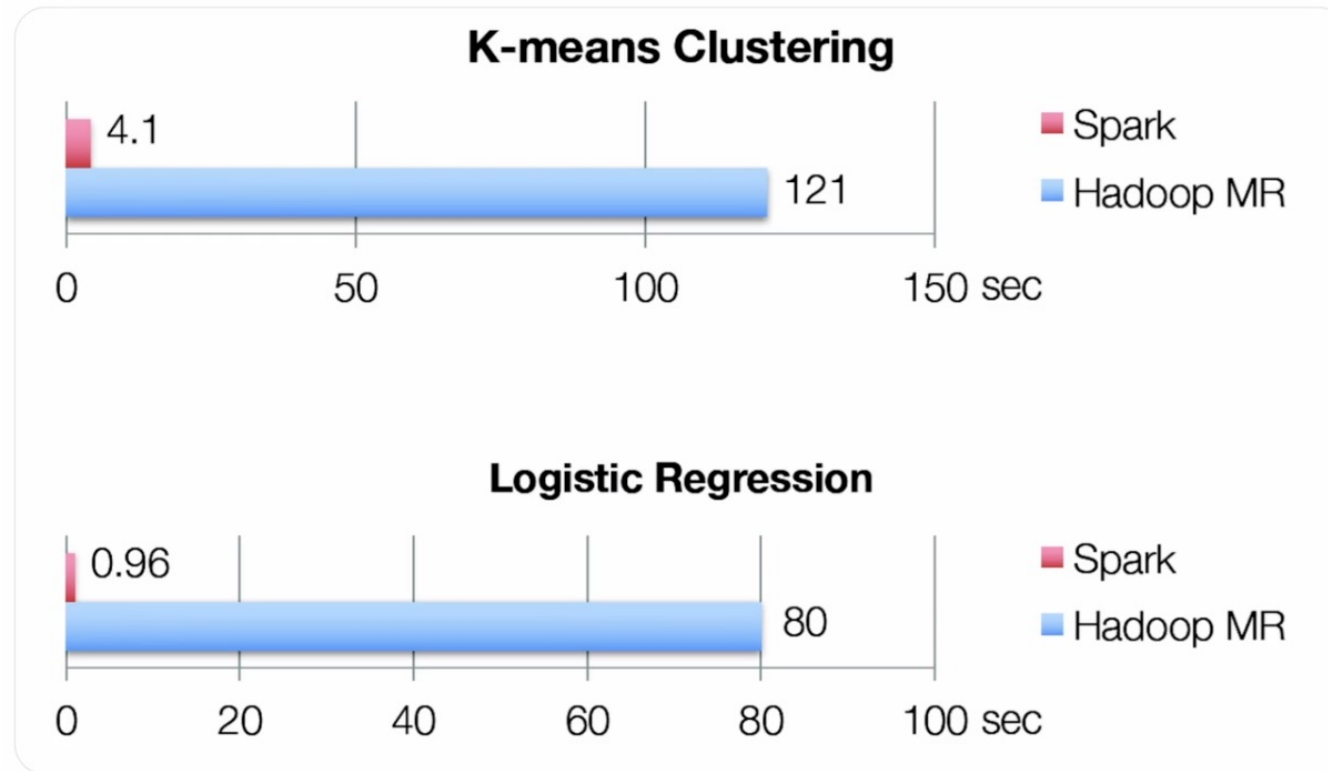
Plan:

- Spark — main abstractions and objects
- Actions and transformations
- Data locality. Serialization

Spark. Motivation



Spark vs MapReduce



Spark vs MapReduce

MapReduce:

- Usually runs slowly
- Mappers write intermediate data to disk before transferring it over the network
- No query optimization
- Sequential jobs must interact with HDFS — bad for iterative tasks
- Not interactive
- Hard to create complex pipelines

```
# Mapper
for line in sys.stdin:
    line = line.strip().lower()
    line = regex.sub(r'[\P{L}]', ' ', line)
    words = line.split()
    for word in words:
        if word != ' ':
            print('%s\t%s' % (word, 1))

# Reducer
word = None
current_count = 0
current_word = None
for line in sys.stdin:
    line = line.strip()
    word, count = line.split('\t', 1)
    if current_word == word:
        current_count += int(count)
    else:
        if current_word:
            print('%s\t%s' % (current_word, current_count))
            current_count = count
            current_word = word
        if current_word == word:
            print('%s\t%s' % (current_word, current_count))
```

Spark:

- Usually runs much faster
- Stores intermediate results in RAM
- Optimizes queries
- Not tied to HDFS
- Can be used interactively
- Easy to program on

```
file = spark.textFile("hdfs://...")

file.flatMap(lambda line: line.split())
    .map(lambda word: (word, 1))
    .reduceByKey(lambda a, b: a+b)
```

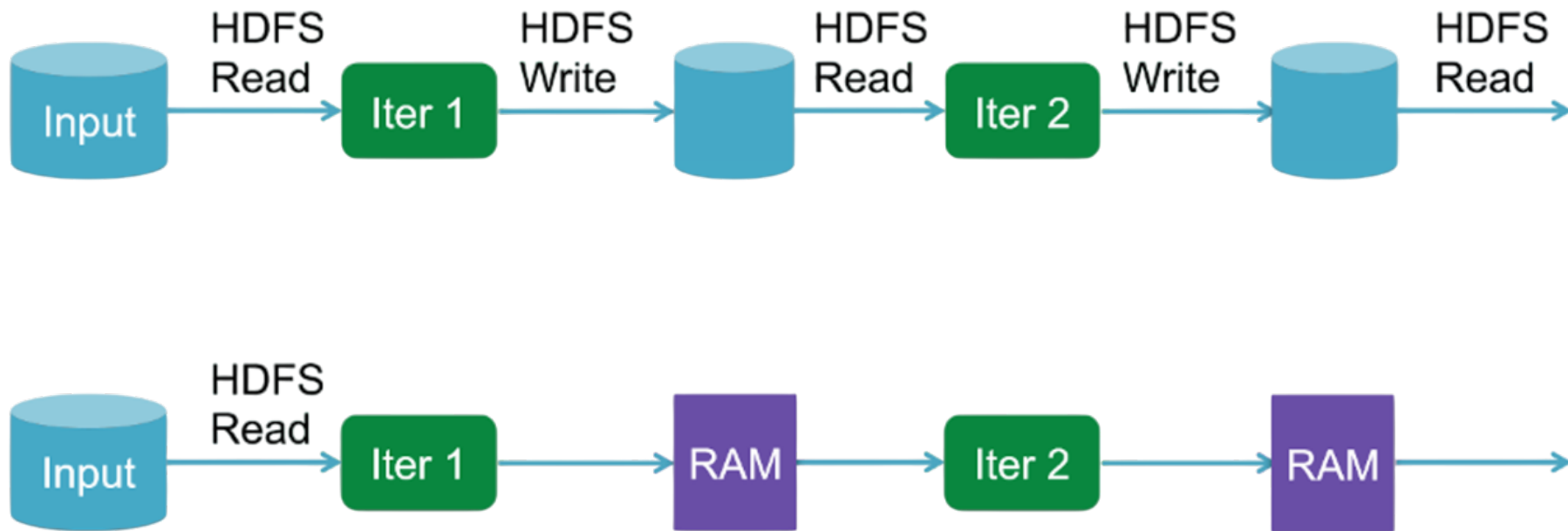
Word count in Spark's Python API

Spark vs MapReduce

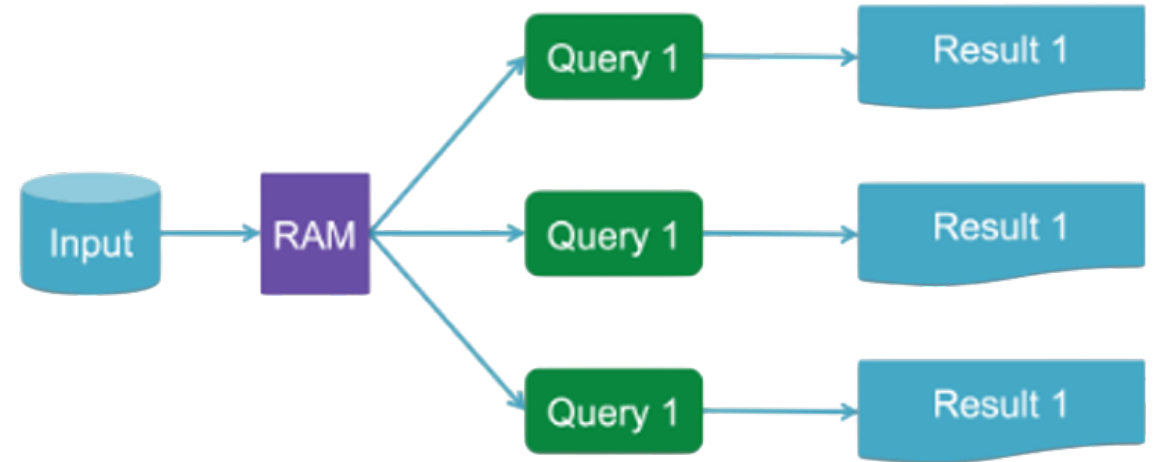
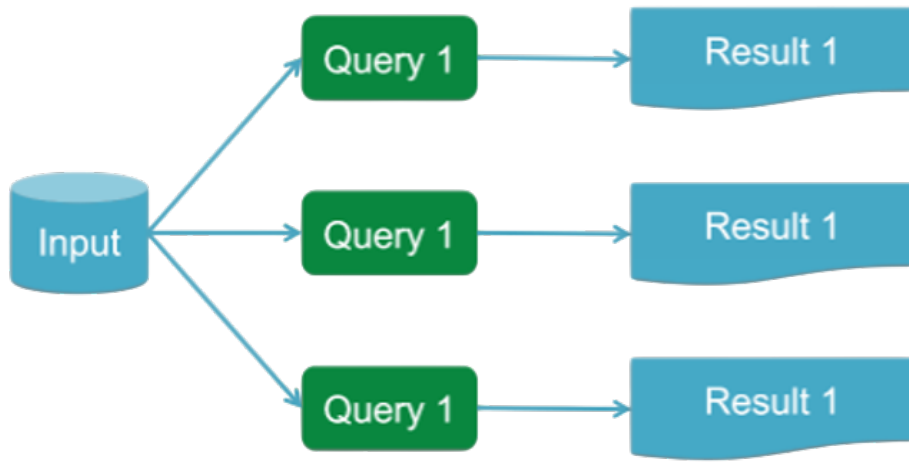
To solve both types of problems, one thing is needed that MapReduce lacks:

- Efficient primitives for data sharing
- In MapReduce, the only way to share data between jobs is through stable storage
- Replication also slows down the system, but it's necessary to ensure fault tolerance

MapReduce vs Spark. Sequential Jobs



MapReduce vs Spark. Multiple Jobs



Spark Outline

History:

- Developed at UC Berkeley's AMPLab in 2009
- Became a top-level Apache project
- <http://spark.apache.org/>

Easy:

- Easy to write applications in Java, Scala, R, and Python
- Supports various data types

Generality:

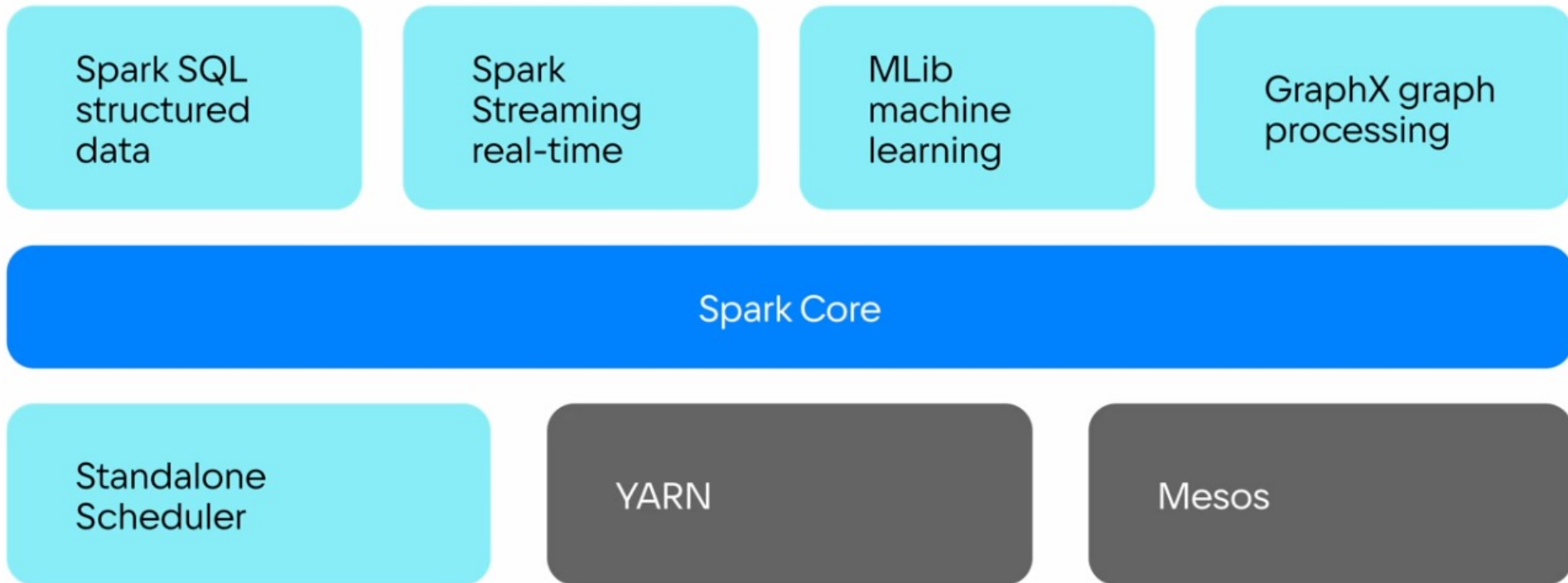
- Combines SQL, streaming, and complex analytics within a single application
- Spark SQL, MLlib, GraphX, and Spark Streaming

Runs anywhere:

- Hadoop, Mesos, standalone, or in the cloud
- Accesses data from various sources (HDFS, Cassandra, HBase, S3)



Spark Components



Spark Architecture

Application:

- A user application written in Spark. Consists of a driver and executors.

Driver program:

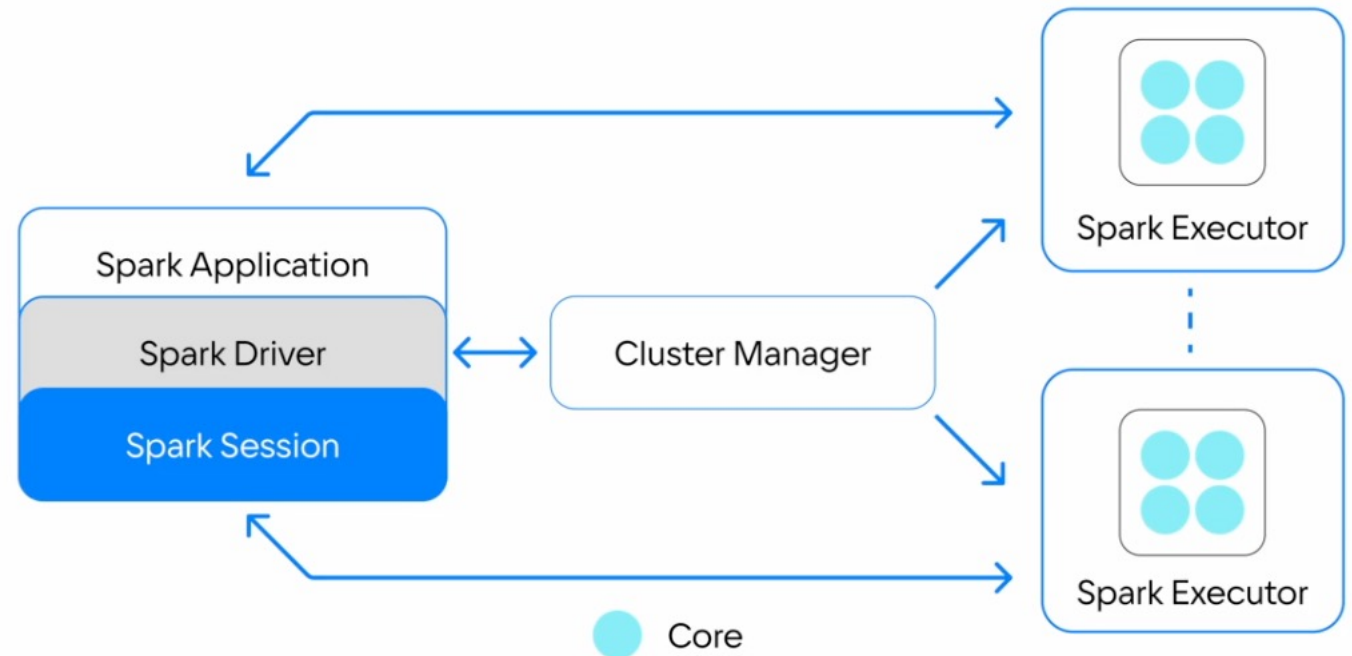
- The program containing the *main()* function that creates and runs the *SparkContext*.

Deploy mode:

- In *cluster* mode, the driver runs inside the cluster. In *client* mode, the driver runs outside the cluster.

Executor:

- A process launched on a worker node to execute the application.



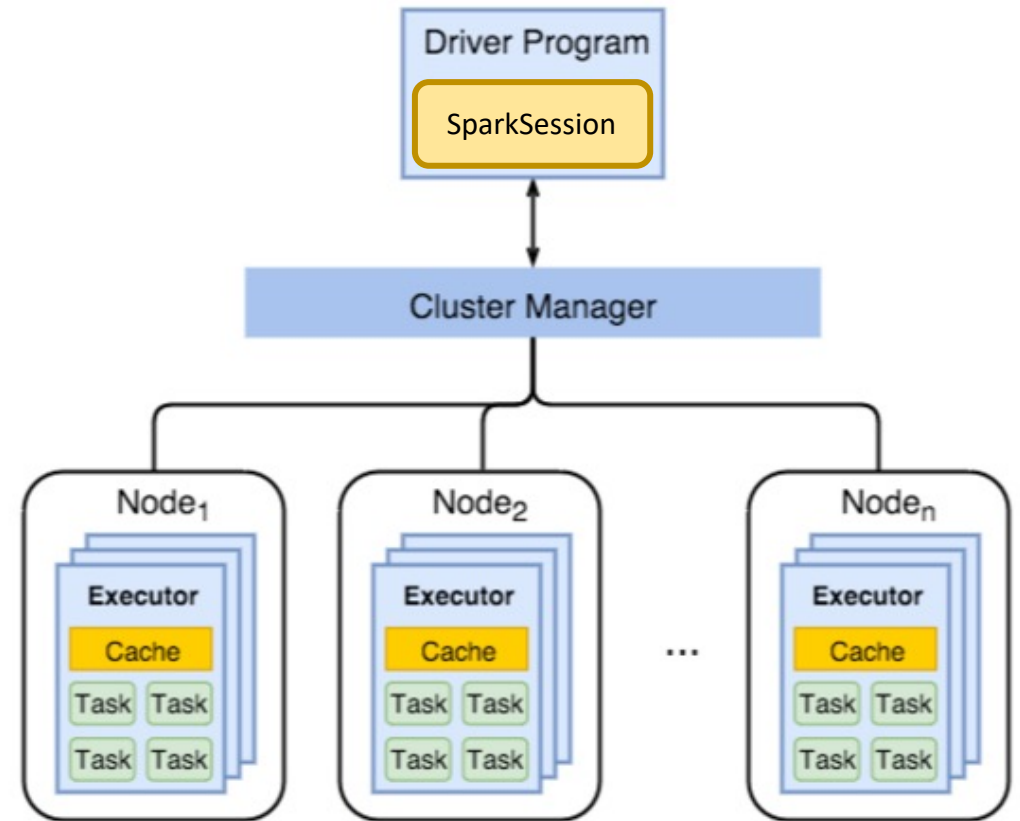
Spark Workflow

Driver:

1. Initializes the application
2. Requests resources
3. Builds the physical plan
4. Sends serialized task code
5. Monitors task execution
6. Finalizes the application
7. Requests resource release

Executors:

1. Receive specific tasks from the driver
2. Send execution status back to the driver
3. Terminate execution upon the driver's command



SparkSession

Unified Entry Point:

- A single point of interaction for various Spark APIs

Configuration Management:

- Allows for the configuration of Spark properties, such as application name, master URL, and memory settings.

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName(
    'abc'
).getOrCreate()
df = spark.read.csv('filename.csv', header=True)
```



Spark Runtime

Task

- The smallest unit of work that will be sent to an executor

Job

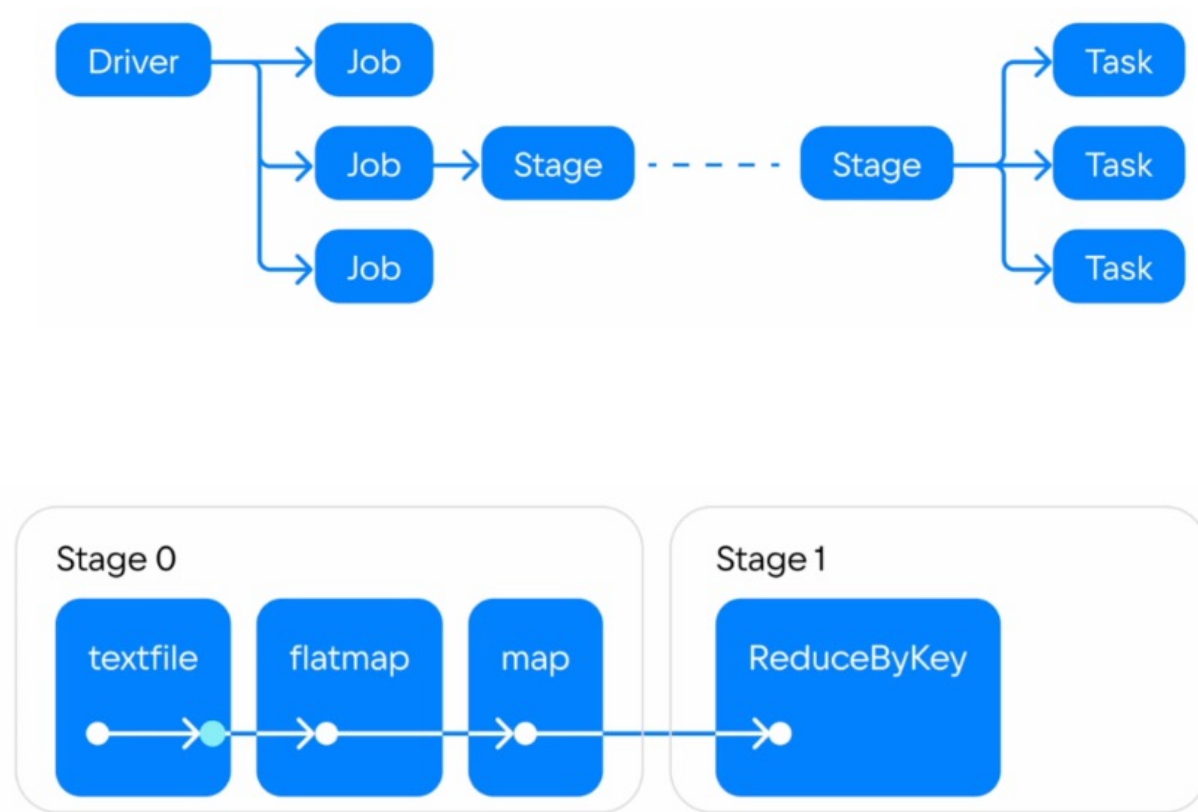
- A parallel computation consisting of multiple stages, triggered by an *action* in Spark (e.g., save, collect)

Stage

- A set of tasks into which each job is divided (similar to Map and Reduce phases in MapReduce)

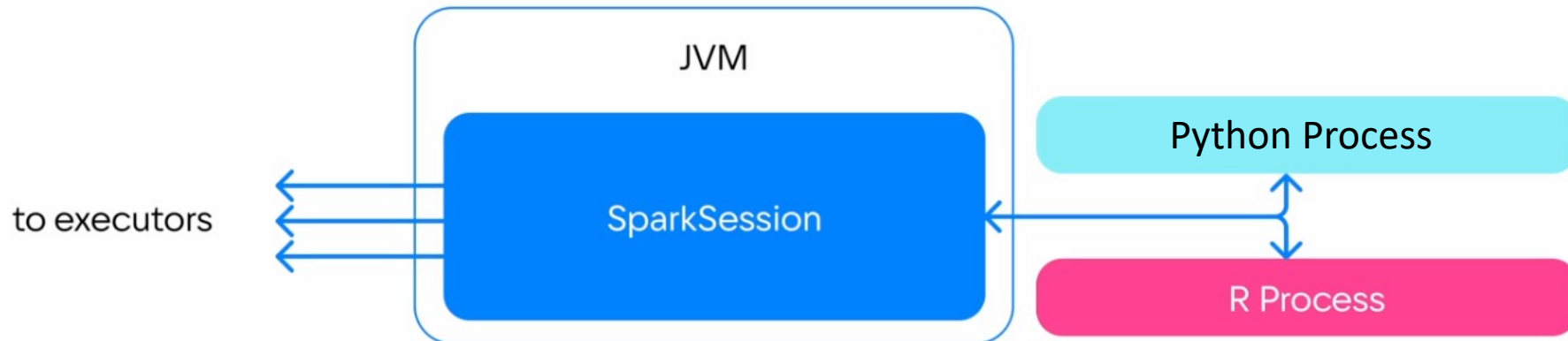
Driver

- Coordinates execution, schedules jobs, and manages task distribution

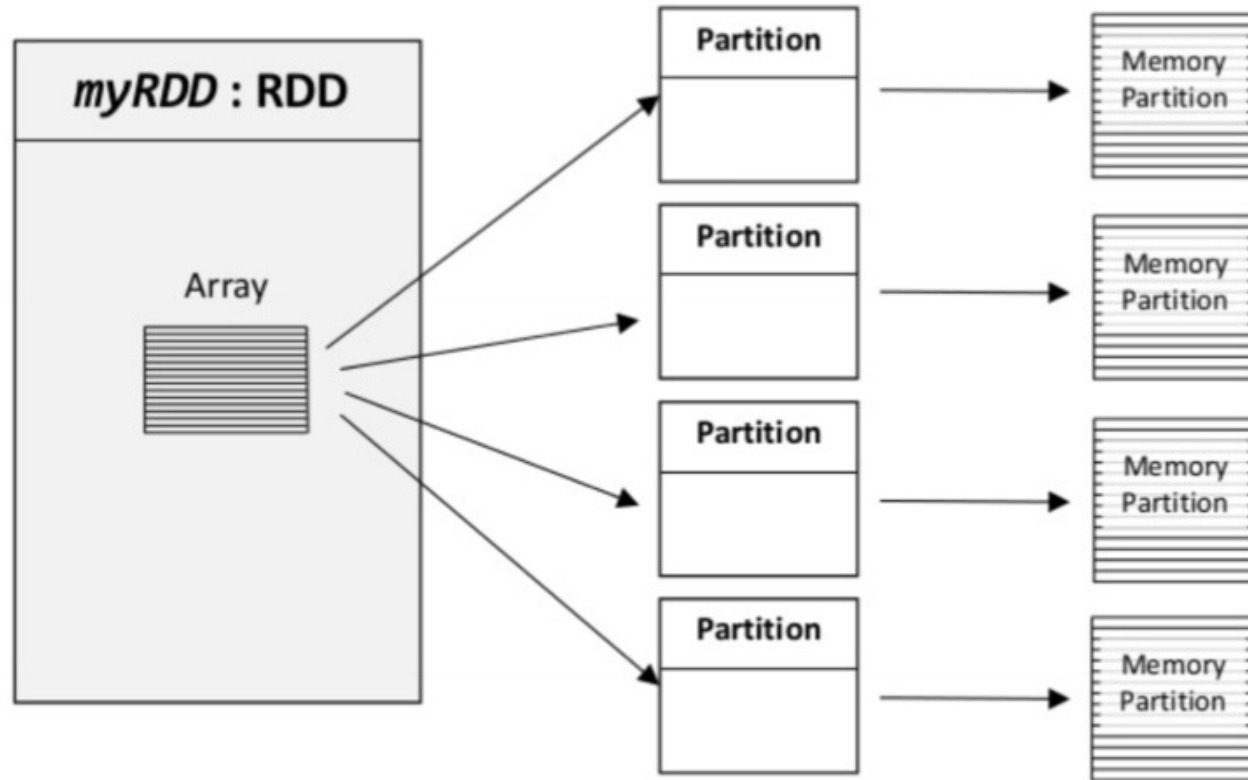


Spark External Processes

- Spark supports Java and Scala natively
- Python and R communicates with the JVM to transfer data:
 - Slow for User Defined Functions
 - Slow for large *actions* outputs



Spark. RDD API



Some RDD Characteristics

- Hold references to Partition objects
- Each Partition object references a subset of your data
- Partitions are assigned to nodes on your cluster
- Each partition/split will be in RAM (by default)

Spark. RDD API

RDD (Resilient Distributed Dataset) — an **immutable** distributed collection of objects

It has three key characteristics:

1. Dependencies

- Tracks how this RDD depends on other RDDs for fault tolerance
- Stores **Lineage Graph**

2. Partitions (with some locality information)

- Can create distributed collections from local FS, HDFS, Cassandra, HBase, Amazon S3, etc.
- Can be stored on different nodes

3. Compute function: Partition => Iterator[T]

- Spark supports text files, SequenceFiles, Hadoop InputFormats, ...

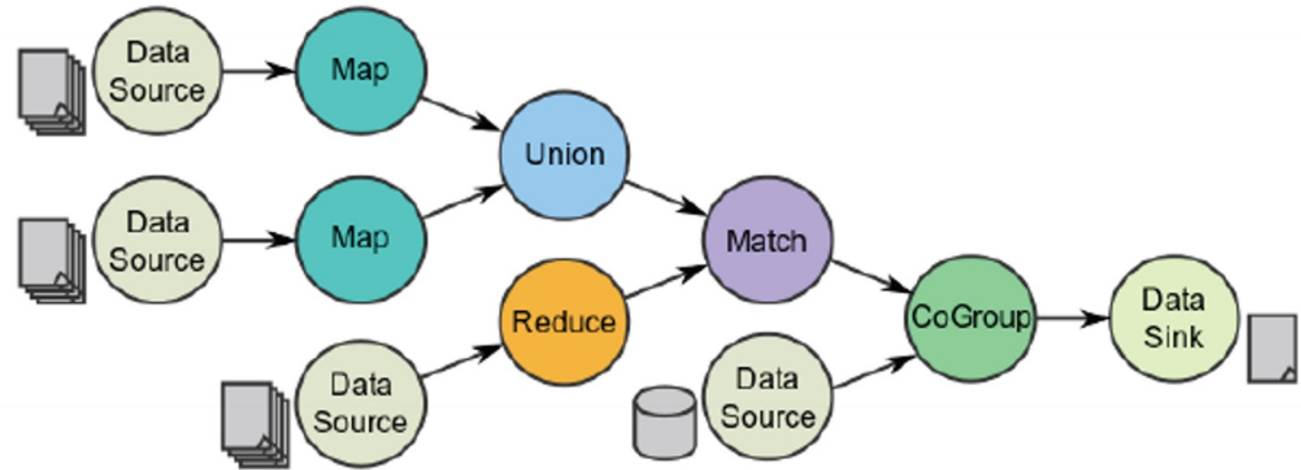
Spark. Programming Model

- Based on **parallelizable operators**
- These operators are higher-order functions that execute user-defined functions in parallel.
- A data processing pipeline consists of any number of **data sources, operators, and data sinks** by connecting the outputs of some operations to the inputs of others.

Spark. DAG

The task is defined using the **directed acyclic graphs (DAG)**:

- Vertices represent RDDs
- Edges represent the operation to be applied to the RDD
- Each edge points from an earlier to a later stage
- DAG scheduler splits the graph into stages and tasks



Spark. High-Order Functions

There are two types of operators:
transformations and **actions**.

Transformations:

- Lazy operators that create new RDDs
- Perform transformations of the form
RDD → RDD

Actions:

- Trigger computation and return results to the program or write data to external storage
- Performs transformations of the form
RDD → actual data

Transformations

```
map(func)
flatMap(func)
filter(func)
groupByKey()
reduceByKey(func)
mapValues(func)
sample(...)
union(other)
distinct()
sortByKey()
...
```

Actions

```
reduce(func)
collect()
count()
first()
take(n)
saveAsTextFile(path)
countByKey()
foreach(func)
...
```

Spark. Lazy Evaluation

The Spark programming model is based on **lazy evaluation**:

- At the moment of job execution, **transformations** on a specific RDD are not performed until at least one **action** is called on it

Spark doesn't store results of the transformations by default:

- Use transformations *cache()* or *persist()* to store the data in memory or on disk after it's computed

RDD API. Wide & Narrow Transformations

There are two types of dependencies between RDDs:

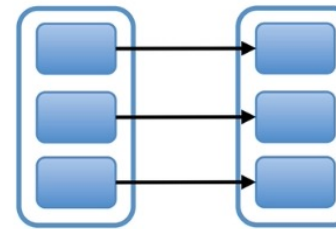
Narrow dependencies:

- Each partition of the parent RDD is used by at most one partition of the child RDD
- Allow pipelined execution on a single cluster node.
 - Example: a filter followed by a map.

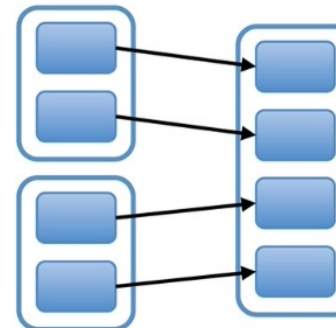
Wide dependencies:

- Each partition of the parent RDD is used by multiple partitions of the child RDD
- Usually requires shuffling across nodes
- Defines the boundary between Stages

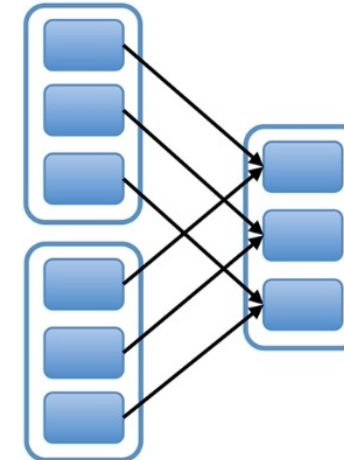
Narrow Dependencies:



map, filter

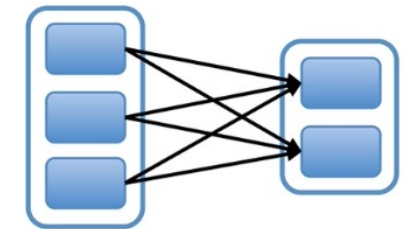


union

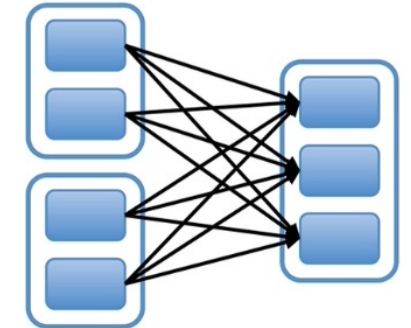


join with inputs
co-partitioned

Wide Dependencies:



groupByKey



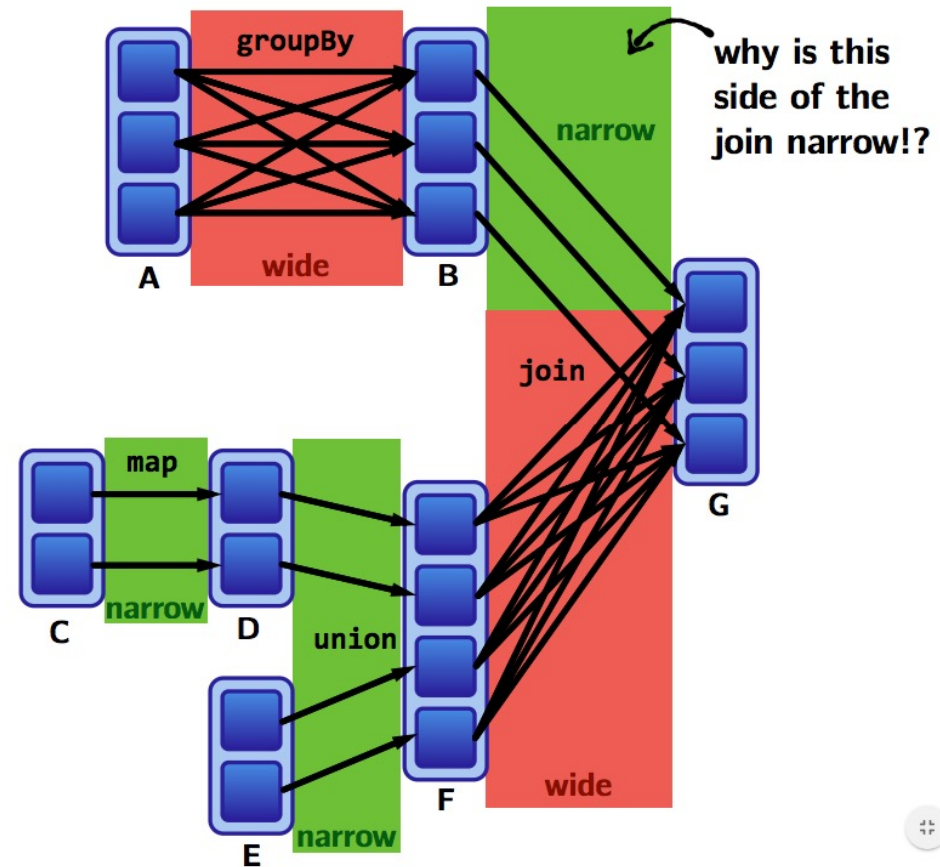
join with inputs not
co-partitioned

RDD API. Wide & Narrow Transformations

Let's visualize an example program and its dependencies.

Wide transformations:
groupBy, join

Narrow transformations:
map, union, join ⚠

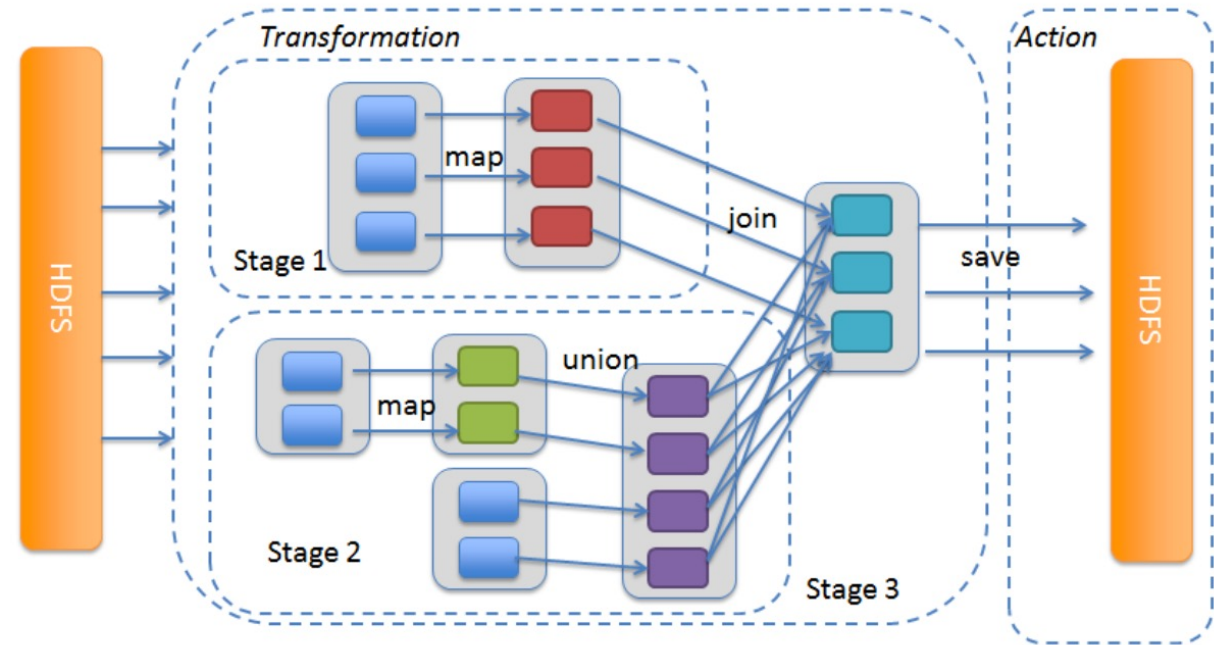


Use `RDD.dependencies` or `RDD.toDebugString` to analyze

Spark. Job Scheduling

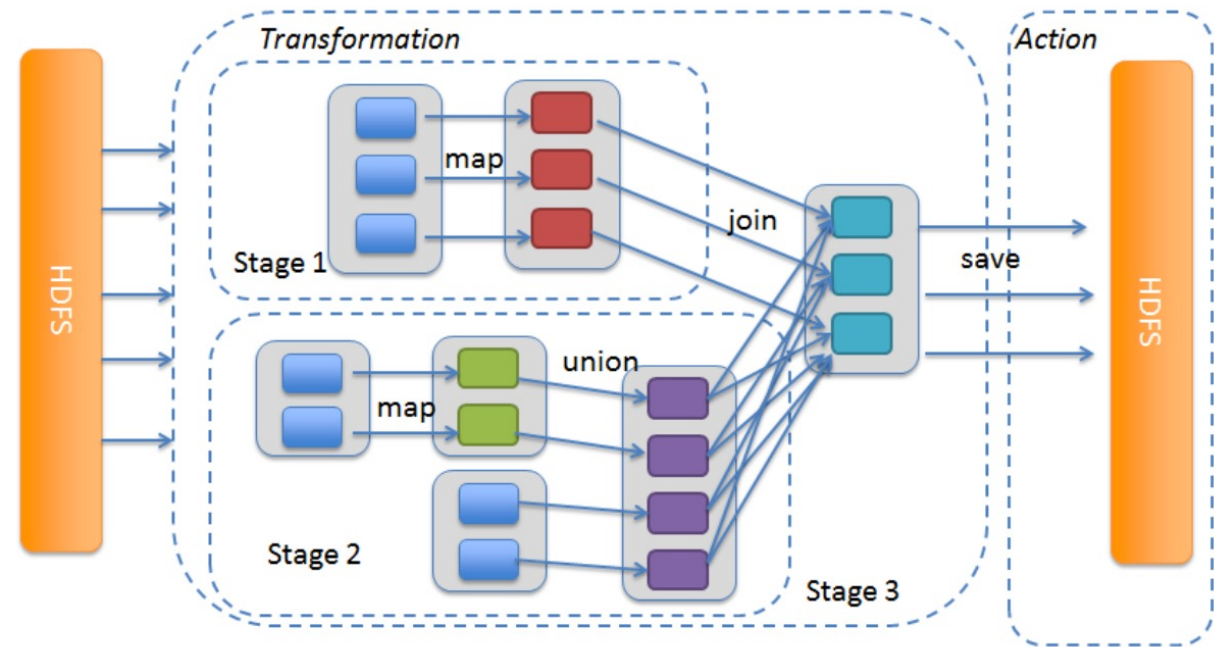
When a user triggers an action on an RDD, the scheduler builds a DAG from the stages of the **RDD Lineage Graph**:

- A stage contains various pipelined transformations with narrow dependencies.
- Boundaries for a stage are defined by:
 - Shuffles for wide dependencies
 - Partitions that have already been processed

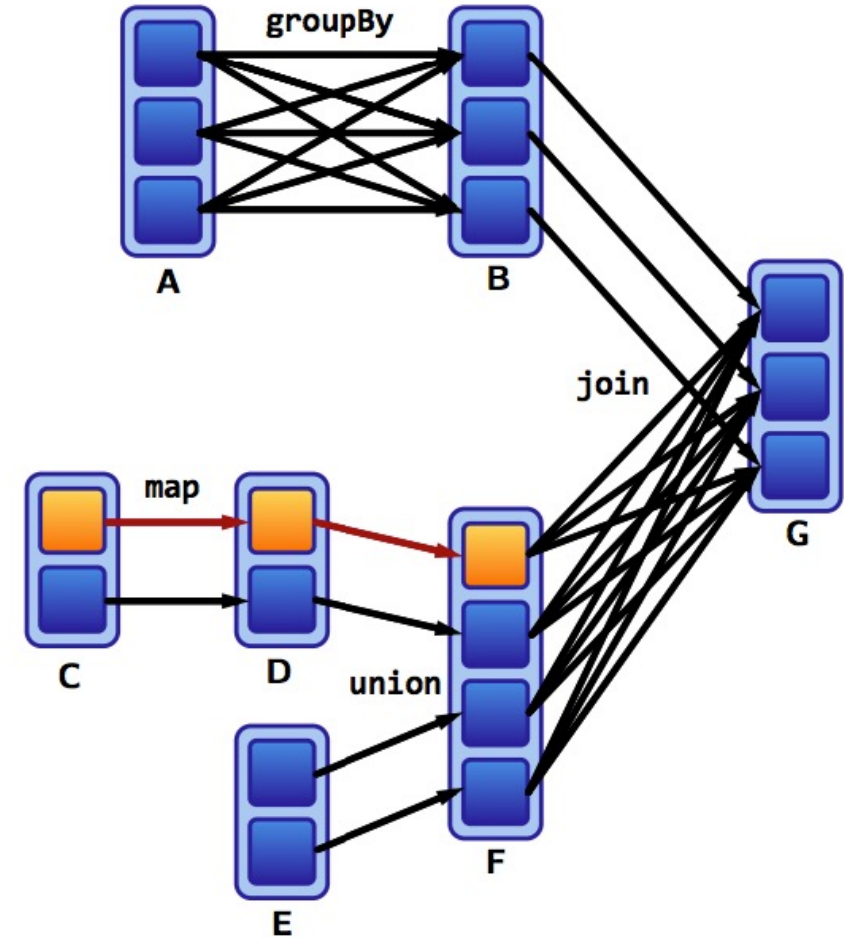
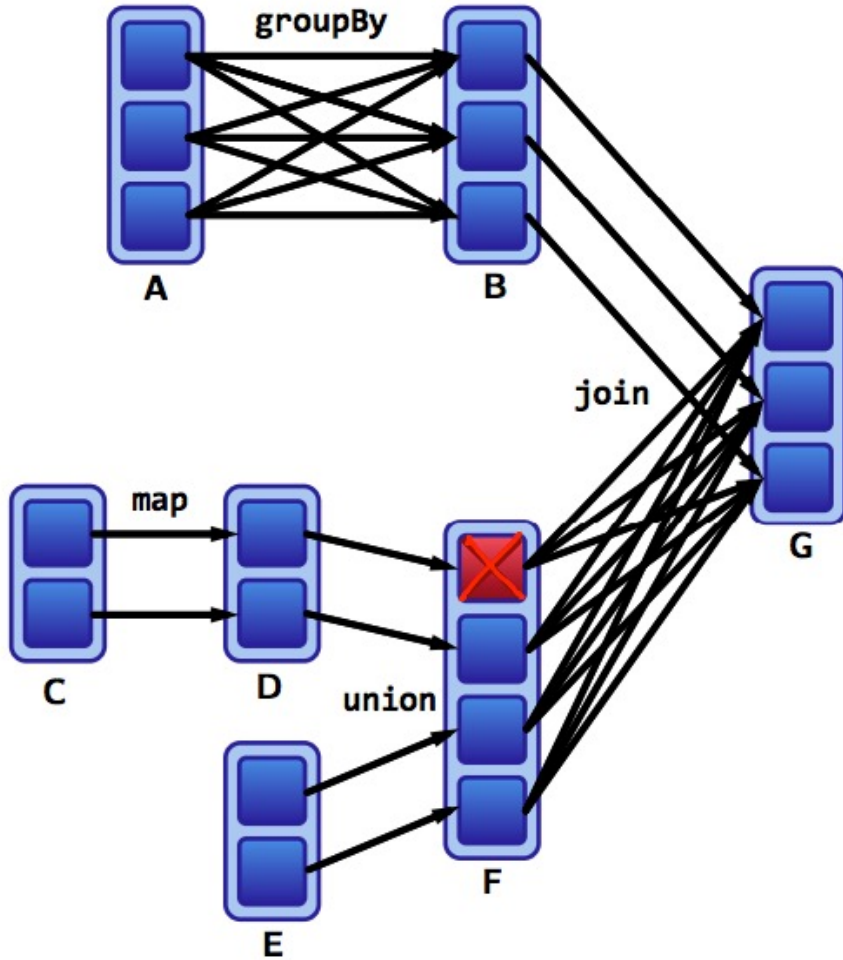


Spark. Job Scheduling

- The scheduler launches tasks to process the remaining partitions (missing partitions) from each stage until the target RDD is computed
- Tasks are assigned to machines based on **data locality**
 - If a task requires a partition that is available in memory on a node, the task is sent to that node



Spark. Fault Tolerance



Spark. Fault Tolerance

RDD maintains lineage information, which can be used to recover lost partitions:

- Intermediate results from wide dependencies are materialized on nodes responsible for the parent partitions to simplify fault recovery
- If a task fails, it will be restarted on another node as long as parent stages are available
- If some parent stages become unavailable, tasks are submitted to recompute the missing partitions in parallel
- Recovery can be time-consuming for RDDs with long lineage chains and wide dependencies
- It may be useful to persist certain RDDs to reliable storage
- The decision of what to persist is left to the developer

Spark. Memory Management

If there is not enough memory for new RDD partitions, LRU (least recently used) eviction is used:

Spark provides three options for storing persistent RDDs:

- In-memory storage as deserialized Java objects
- In-memory storage as serialized Java objects
- On-disk storage

Spark. Memory Management

For persistent RDDs, each node keeps any RDD partitions in RAM, allowing new actions to execute much faster

This is done using the `persist()` or `cache()` methods

Different storage levels:

- `MEMORY_ONLY`
- `MEMORY_AND_DISK`
- `MEMORY_ONLY_SER`
- `MEMORY_AND_DISK_SER`
- `MEMORY_ONLY_2`, `MEMORY_AND_DISK_2`, etc.

Spark. Shared Variables

Sometimes it is necessary for a variable to be shared between tasks or between tasks and the driver program

Using regular read-write shared variables between tasks is inefficient:

- Read-only — Broadcast Variables
- Write-only — Accumulators

Spark. Broadcast Variables

- **Read-only variables** are cached on each machine instead of sending a copy to every operation
- **Broadcast Variables** are sent to a node only once

```
broadcasted_value = spark.sparkContext.broadcast(value)
```

```
...
```

```
# Inside a function
```

```
broadcasted_value
```

```
...
```

```
# Remove the value from the nodes
```

```
broadcasted_value.unpersist()
```


Spark. Accumulators

- Variables that can only be incremented
- Can be used to implement counters and accumulators
- Tasks running on the cluster can then add values using the += operator

```
accumulator = spark.sparkContext.accumulator(0)

...
# Inside a function
accumulator.add(1)
...

print(accumulator.value)
```

Spark. Use cases

Applications suitable for RDDs:

- Batch applications that apply a single operation to all elements in a dataset

Applications not suitable for RDDs:

- Applications performing asynchronous fine-grained updates, modifying shared state (e.g., storage systems for web applications)